

Assignment 3 — Case Study 3 – Project AURELIA: Automated Financial Concept Note Generator

Background

In this case study, you will build AURELIA, a production-grade microservice that automatically generates standardized concept notes for financial topics. The system will extract and process content solely from the provided Financial Toolbox User's Guide (fintbx.pdf), applying a Retrieval-Augmented Generation (RAG) approach for structured concept synthesis. When a queried concept cannot be located within the PDF corpus, the system will fallback to retrieving content from Wikipedia to ensure coverage.

Learning Objectives

- Design an end-to-end RAG pipeline using LangChain for chunking, Pinecone or ChromaDB for vector storage, and Airflow for orchestration.
- Build a FastAPI microservice exposing endpoints for both retrieval and structured concept-note generation.
- Implement a Streamlit frontend that queries the service, displays retrieved/seeded concepts, and allows on-demand generation.
- Construct a concept database seeding pipeline that uses the instructor package to create structured JSON entries for concepts, cached for reuse.
- Integrate Wikipedia fallback retrieval when a concept is not found in the fintbx.pdf corpus or the local concept database.

System Overview

Deployment Requirement: The entire application must be deployed on a managed cloud platform (GCP or AWS). No local components are allowed, including Airflow, databases, and application hosting (use simple managed hosting such as Cloud Run, App Engine, or AWS EC2/Elastic Beanstalk). Ensure all services run in the cloud and can be demonstrated end-to-end.

Component	Description
PDF Parser & Chunker	Parses fintbx.pdf using a parser of your choice. Ensure extraction captures all code snippets, images/figures, formulas/equations, and body text, preserves reading order, and retains figure/table captions for accurate citations.
Vector Store (Pinecone/ChromaDB)	Embeddings are stored in a Pinecone or ChromaDB index (namespaced by topic)

	headings in the Financial Toolbox).
Concept DB Seeder Pipeline	Airflow DAG that runs on a list of finance concepts (e.g., 'Duration', 'Sharpe Ratio', 'Black-Scholes'), generates structured JSON using instructor, and stores results
RAG Service (FastAPI)	Provides endpoints /query and /seed that retrieve, generate, or update concept notes. Uses the PDF corpus as the primary source.
Streamlit Frontend	Enables users to input financial concepts, view cached concept notes, or trigger new note generation via the FastAPI backend.
Fallback Layer (Wikipedia)	If RAG retrieval from fintbx.pdf yields no relevant context, Wikipedia content is fetched and standardized through the instructor model to maintain the note structure.

Assignment Structure & Labs

Lab 1 — PDF Corpus Construction

1. Parse fintbx.pdf using a parser of your choice. Ensure the extraction captures all code snippets, images/figures, formulas/equations, and body text, preserves reading order, and retains figure/table captions for accurate citations.
2. Use LangChain for chunking, but experiment with multiple strategies (e.g., RecursiveCharacterTextSplitter, MarkdownHeaderTextSplitter, section/heading-aware splits, code-aware splits) and vary chunk sizes/overlaps. Justify your chosen strategy with examples and retrieval metrics.
3. Embed each chunk using text-embedding-3-large.
4. Store embeddings with metadata (section titles, page numbers) in Pinecone or ChromaDB.

Lab 2 — Managed Orchestration in the Cloud

5. Choose GCP (Cloud Composer) or AWS (Amazon MWAA) — local Airflow is not permitted.
6. Create two DAGs: fintbx_ingest_dag and concept_seed_dag.
7. Configure cloud storage for artifacts (GCS buckets or Amazon S3)
8. Parameterize schedules for weekly refresh and on-demand seed updates.

Lab 3 — FastAPI RAG Service

9. Implement /query and /seed endpoints.
10. Query Pinecone or ChromaDB for top-k similar chunks.

11. If no context is found, trigger Wikipedia fallback retrieval.
12. Pass context to the instructor model to produce a structured note.
13. Store or update the note in Postgres for caching.

Lab 4 — Streamlit Interface (Cloud Hosted)

14. Host Streamlit on a simple managed service (Cloud Run, App Engine, or AWS EC2/Elastic Beanstalk).
15. Develop a frontend to input a concept query.
16. If cached, display the note with PDF section references.
17. If not cached, trigger FastAPI /query to generate the note.
18. Include clear indication when Wikipedia content is used.

Lab 5 — Evaluation & Benchmarking

19. Evaluate concept note quality using accuracy, completeness, and citation fidelity.
20. Compare generation latency for cached vs. newly generated notes.
21. Report retrieval latency and token cost using text-embedding-3-large for both Pinecone and ChromaDB configurations.

Deliverables

- GitHub Repository with all pipeline, API, and frontend code.
- Cloud Deployment on GCP or AWS showing all services operational.
- Video Demo (5–8 minutes) demonstrating end-to-end cloud execution.
- Codelab Document explaining architecture, endpoint flow, fallback logic, and cloud deployment.
- README.md with setup instructions and architecture diagram.
- Attestation Statement confirming originality of work.

Evaluation Rubric

Category	Weight	Criteria
Cloud Deployment	25%	Fully cloud-hosted application (GCP or AWS). No local execution. Uses managed Airflow, storage, and hosting.
Design Architecture	20%	PDF-first RAG with Wikipedia fallback, clean service boundaries, and dual vector DB support (Pinecone/ChromaDB).
Pipeline Automation	15%	Properly scheduled cloud DAGs, artifact management, and error handling.
Concept DB & Instructor Integration	15%	JSON schema compliance, regeneration, and freshness logic.
Frontend UX & API Integration	10%	Functional Streamlit UI with API connectivity and clear fallback indicators.
Evaluation & Benchmarking	10%	Performance metrics, latency, and embedding retrieval analysis.
Documentation & Reproducibility	5%	Clear cloud setup instructions, diagrams, and reproducibility.